

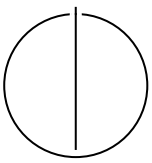
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics in Informatics

Flat Memory Mode for CXL GPUs

Ivan Logvynenko





SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

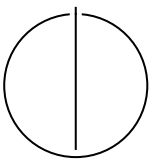
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics in Informatics

Flat Memory Mode for CXL GPUs

Flache Speicher Modus fur CXL GPU

Author:	Ivan Logvynenko
Examiner:	Prof. Dr. Phamod Bhatotia
Supervisor:	Dr. Anatole Lefort
Submission Date:	01.10.2026



Abstract

In the past years a lot of advancements were made in order to allow GPU to cope with heavier and heavier workloads. On the other hand, current demands in compute power also continue to grow, and such demand almost always outpaces the technological advancement, thus making a research in optimizing the technology that already exists to fit modern use cases quite important in our world. In this paper one such optimization will be discussed, as how to increase available GPU memory and mitigate possible arising issues of such optimization.

Contents

Abstract **ii**

1 Introduction **1**

2 Background **4**
2.1 GPU programming model 4
2.1.1 Dedicated memory 4
2.1.2 Unified memory 5
2.2 GPU memory expansion 6
2.2.1 NVLink 6
2.2.2 Copy-then-execute 6
2.2.3 UVM 7
2.3 CXL 8
2.4 CXL for GPU memory expansion 8
2.5 Comparison 9

3 Design **10**
3.1 The Problem 10
3.2 The Proposal 10
3.3 Design goals 11
3.4 Challenges 12
3.4.1 CXL client integration 12
3.4.2 Design of cache hierarchy 13
3.4.3 Guaranties of coherency and consistency of CPU and GPU sides 13
3.5 Architecture overview 13
3.6 System workflow 14
3.6.1 Store 14
3.6.2 Load 14
3.7 Summary 14
3.8 Technical background 15

4 Implementation **16**
4.1 Basic operations . . . 16
4.1.1 Store . . . 16
4.1.2 Load . . . 17
4.2 Asynchronous write-back . . . 18
4.3 Address translation . . . 18
4.4 Automatic data balancing . . . 19
4.5 Module integration . . . 19

5 Evaluation **21**
5.1 Research question . . . 21
5.2 Evaluation Setup and Methodology . . . 21
5.3 Benchmark suite: Chai . . . 22
5.4 Performance comparisons . . . 22
5.5 Cached and uncached system performance . . . 23
5.6 Baseline comparison . . . 25
5.6.1 Change of the benchmarking suite . . . 25
5.6.2 Rodinia benchmarks . . . 26
5.6.3 Benchmark results . . . 26

6 Summary and conclusion **28**

7 Related work **29**
7.1 Disaggregated, Vendor-Generic APUs with CXL . . . 29

8 Future work **30**
8.1 Realistic workloads . . . 30
8.2 Newer versions of ROCm . . . 30

Abbreviations **31**

List of Figures **32**

List of Tables **33**

Bibliography **34**

1 Introduction

Data parallel workloads. Modern workloads need to process hundreds of gigabytes of data, which makes them both compute and data intensive. Such workloads come from different backgrounds such as scientific researches and computations [17], data analysis and business forecasting[16], or deep learning and Ai model inference.[3][6][2] All those workloads have a common feature that they are prone to be embarrassingly parallel[8]. Consequently, Central Processing Unit (CPU) is not the best tool to compute them, because CPU is optimized for low latency and executing one stream of instructions as quickly as possible. It is also optimized for control-flow heavy programs whereas these workloads mainly do mathematical computations. What would be the best then? The answer is Graphical Processing Unit (GPU). GPU is both optimized for high data parallelism and would prefer workloads that do not do much control flow operations. [13]

General-purpose computing on GPUs (GPGPU). Originally, GPUs were fixed-function devices designed for a single purpose: rendering graphics, where thousands of pixels are processed independently and in parallel. Over time, however GPU started to be treated not as a graphics device but as a general throughput-oriented co-processor. In this model, a computation is expressed as a kernel that is executed by thousands of lightweight threads simultaneously, each applying the same instructions to different data elements. [14]

GPU and Ai workloads. Modern day Ai workloads are demanding more and more from GPUs. The main resource that is important is the memory. For instance: GPT-2, which was released in 2019, had 1.5 billion parameters.[10] In 2022, GPT-3 was released and it had 175 billion parameters. [9] In order to store 1 parameter 2 bytes of memory are allocated, then in order to load such a model a system needs to have at least 350 GB of memory. The latest model by DeepSeek, DeepSeek v3, released in 2025, has 671 billion parameters [7], consequently starting such model would require around 1.34 TB of memory. Only top tier GPUs can offer such capacity

GPU memory capacity expansion. There are multiple opportunities to expand the memory of the system: *NVLink*, *copy-then-execute* and *UVM*. The NVLink expands memory by allowing for efficient GPU interconnection, thus GPUs can exchange data via low latency peer-to-peer connections. [15] The copy-then-execute model utilizes the

host memory to expand the GPU memory. The data that has to be loaded is splitted into chunks and two chuck are swapped on demand. Unified Virtual Memory (UVM) is a software solution that is also using the host memory to expand GPU memory, as in the copy-then-execute, however, instead of swapping large chunks of data, in the UVM model each GPU memory request is tracked and if the address in the request is not present in the local memory, it would be fetched from the host memory. [20]

Problems of existing solutions. Each of the solutions described has it's own tradeoffs. For example, in order to be able to expand the memory by the usage of *NVLink*, one has to own multiple GPUs that they could interconnect. With the ongoing memory crisis, GPUs costs are also rising, which means that this approach is hardly affordable or rentable. In the *copy-then-execute* model the problem lies in the moment when data chunk has to be swapped. This operation is taking time, which makes GPU idle. Consequently, the total latency increases gradually. Moreover, the programmer has to decide how to split the data so that it will not be swapped often, because otherwise, constant block swapping will cause a graet decline in performance. Lastly, the *UVM* model is not optimal too. Unlike copy-then-execute, data fetching will be happening more often than block swapping. Every such fetch, not only contributes to the latency, but it also consumes the bandwidth.

Solution requirements — Research gap. With all the problems described earlier it is easy to conclude that there a need for system that would be:

- **Cheap** enough to be affordable
- **Performant** as the end users would prefer systems with less latency
- **Little to no complexity** in the use by the programmers

Use of CXL. The technology that answers those needs best is the CXL. CXL allows for interconnection of devices, one of the features being memory expansion. CXL memory uses DDR memory, consequently it is definitely cheaper then the GPU memory. Even though CXL memory uses the same hardware connection, PCIe, as is between GPU and host memories, it has it's own protocol making it far more performant than data transfers over PCIe. Lastly, CXL memory is cache-line accessible from the GPU, hence the programmers get an easy way to create their applications. [1]

CXL issues. On the other hand, CXL has it's disadvantages. To begin with, compared to the GPU local memory, CXL memory has far less bandwidth and noticeably higher latency. Hence each memory access contributes to the total latency of the application. Consequently, in order to speedup applications, programmers would still issue a data copying from the CXL memory to the GPU memory, because accessing data in the GPU memory from the GPU is faster. Lastly, the programmers are left with choice: what

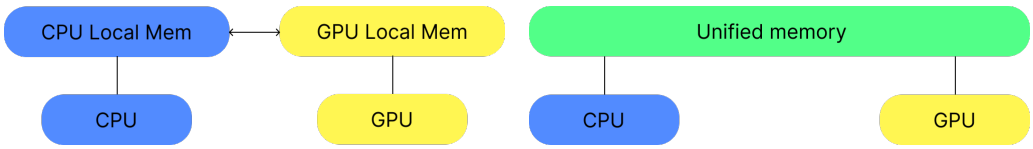
data to copy and what not to? The incorrectly selected moved data would occupy the precious GPU memory, and the other data which was not moved, but should have been would be requested each time from the slower CXL memory.

Use GPU memory to mitigate CXL issues. Since, GPU memory is better then the CXL memory, we are going to automate it's usage and take off the decision responsibility from the programmer. We are going to create a hardware expansion that uses the GPU memory to store the data from the CXL to avoid further CXL requests, thus effectively reducing latency per memory request. This way CXL memory would be a great option to expand scarce GPU memory, without introducing it's own tradeoffs.

2 Background

2.1 GPU programming model

There are two types of GPUs: GPUs with *dedicated* and *unified* memory.



(a) Dedicated memory (b) Unified memory

2.1.1 Dedicated memory

Hardware architecture. Dedicated memory is memory that is soldered near the GPU and is therefore optimized to be accessed primarily by the GPU. In a dedicated GPU architecture, the GPU and CPU have separate memory spaces: the CPU accesses system memory, while the GPU accesses its dedicated memory, commonly referred to as VRAM. Communication between these two memory spaces is typically performed over the PCIe bus. Although PCIe provides a high-bandwidth connection, its bandwidth and latency are considerably worse than the GPU's dedicated memory, consequently, transferring data between system memory and VRAM can become a performance bottleneck. [12]

Performance characteristics. Virtual Random Access Memory (VRAM) provides higher bandwidth and lower latency than unified memory when accessed from the GPU. This allows the GPU to access the data required for computation once it has been transferred to VRAM. However, data transfers between system memory and VRAM are performed over PCIe and are more expensive than accesses to memory that is already available to the GPU. In particular, frequent small transfers can introduce a noticeable overhead due to the latency and limited bandwidth of PCIe. Therefore, small copies should be avoided where possible, and data should instead be transferred in larger batches.

Programming interface. The separation between CPU memory and VRAM is also reflected in the programming interface. For example, CUDA provides explicit memory-

management functions such as `cudaMalloc()` to allocate memory in VRAM and `cudaMemcpy()` to transfer data between host memory and device memory. Before a GPU kernel can process data stored in CPU memory, the required data must first be copied to VRAM via `cudaMemcpy()` into the memory that was allocated in the GPU memory via `cudaMalloc()`. The GPU can then access the data directly from its dedicated memory during computation. Once the computation has finished, results that are required by the CPU must be copied back from VRAM to system memory using a device-to-host transfer. [12]

2.1.2 Unified memory

Hardware architecture. Unified memory provides a shared memory pool that can be accessed by both the CPU and the GPU. Unlike dedicated memory, where the CPU and GPU have separate physical memory spaces, unified memory allows both processors to operate on the same physical memory. In systems with unified memory, CPU and GPU are connected to the memory through Network-on-chip (NoC), which provides a high-bandwidth interconnect between the processing units and the memory controllers. This architecture eliminates the need to maintain separate copies of that data in CPU and GPU memory, therefore, it avoids explicit transfers between two memories. [11]

Performance characteristics. Unified memory can provide lower memory-access latency for the GPU than accessing dedicated memory through a separate PCIe-connected memory system, since data does not need to be explicitly transferred between CPU and GPU memory. However, the total latency of a memory operation can still be higher compared to the local access to dedicated GPU memory, depending on the architecture and the location of the requested data. The main advantage of unified memory is therefore not necessarily lower raw memory latency, but the ability to provide both processors with direct access to the same memory pool.

Programming interface. The unified memory model also simplifies GPU programming because the CPU and GPU can use the same memory addresses to access shared data. For example, HIP provides `hipMalloc()` for allocating memory that can be accessed by both the CPU and GPU, depending on the underlying system and allocation type. Since the memory is shared, the programmer does not need to explicitly copy data between separate host and device memory regions using operations analogous to `cudaMemcpy()`. In order to access hip-allocated memory, both CPU and GPU would need to deference a pointer returned by the hipMalloc().

Discussion. The unified memory model is mostly used is small, low-performance devices such as phones or embedded chips. This is due to the requirement of the GPU and CPU being on the same motherboard to connect them to the same memory pool,

hence such systems are not capable of showing the same power that is needed by the modern workloads. For this reason, in the data centers one would find a dedicated memory model being used. Even though this makes development of applications considerable more complicated, this is not as important as the end performance of the system.

2.2 GPU memory expansion

There are several approaches that could be taken into consideration as to how expand the available GPU memory:

2.2.1 NVLink

Technology description. NVLink is NVIDIA's proprietary interconnect technology that allows multiple GPUs to be linked together directly. NVLink, provides higher bandwidth and lower communication overhead than conventional PCIe connections. In a multi-GPU system, each GPU still has its own physical memory, but the GPUs can exchange data directly, thus increasing that available GPU memory [15] **Benefit for programmers**. The main benefit of an NVLink-based system, besides easy memory expansion, is the high-bandwidth communication between GPUs. Compared with transferring data between GPUs over PCIe, NVLink can reduce the cost of exchanging data between devices. **Affordability**. The main disadvantage of an NVLink-based approach is its cost. The total price of multi-GPU systems can be prohibitively high, placing NVLink-based computing out of reach for many individual developers, small teams, or budget-conscious organizations, and making it suitable only for well-funded research labs, enterprises, and cloud providers.

2.2.2 Copy-then-execute

Technology description. Copy-then-execute is a software approach in which a developer splits data into slices/chunks and copies data relevant only to the current computation stage to the GPU and then starts computation. During the execution, new relevant chunks are swapped with those, that became irrelevant during the process of computation. This way, more data then there is available GPU memory could be processed.

Example with figure. Consider the situation described by the Figure 2.2. In this example there are 3 chunks that are loaded in the GPU Local memory. When a GPU needs a chunk that is not stored in the memory it would need to request the relevant

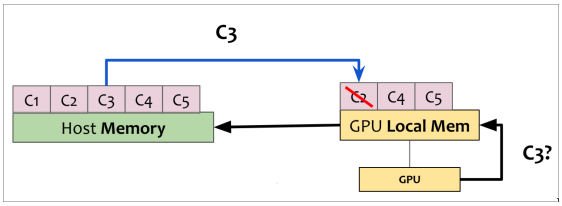


Figure 2.2: Copy-the-execute example

chunk from the Host memory and to replace it with the newly received data. This process takes enough time to put the GPU to idle.

Programming drawbacks. Each application has different relevant chunks and different steps of execution, which mean that creation of a generalized and automatic approach is not possible. Consequently a programmer has to decide, how to split the data and in which order to load it to the GPU. The correctness of data splitting and ordering would have a big impact on the end performance.

2.2.3 UVM

Technology description. Unified Virtual Memory (UVM) is a software solution with the aim of letting a programmer with a GPU that has dedicated memory model, create an application that would work as if unified memory was used. In order to accomplish that, data would still be copied to the dedicated memory, but if GPU requests the data that is not present in the GPU memory, then a request to the CPU memory will be issued and a requested data retrieved. Similarly, data can be sent back to system memory when necessary. Thus, UVM can allow an application to operate on datasets larger than the physical capacity of the GPU's VRAM without requiring the programmer to explicitly manage every individual data transfer. [20]

Flat memory benefits. A definite benefit of such approach is simplicity for the programmer, as this will not require any complex evaluations as in the copy-then-execute approach. Furthermore, the same application can potentially run on systems with different amounts of GPU memory without requiring the programmer to manually redesign the data partitioning strategy.

Performance drawbacks. When the GPU accesses data that is not currently present in VRAM, the memory page, containing the requested data will be moved from system memory to the GPU. On a discrete GPU, such transfers typically occur over PCIe. Although PCIe can provide sufficient bandwidth for large transfers, individual page copies can suffer from high latency. If GPU continiously accesses data that is not present in VRAM, this would cause GPU to idle while waiting for data.

2.3 CXL

CXL description. Compute Express Link (CXL) is an industry-supported Cache-Coherent Interconnect for Processors, Memory Expansion and Accelerators.[1] It is built on top of the existing Peripheral Component Interconnect Express (PCIe) physical infrastructure while providing protocols specifically designed for cache-coherent communication. This makes CXL a promising technology for low-latency memory expansion in heterogeneous computing systems. Unlike conventional GPU memory expansion, where data has to be explicitly transferred between system memory and dedicated GPU memory, CXL enables devices to access an external memory pool while maintaining memory coherency.

- **CXL.io** - is an IO sub-protocol based on the PCIe. It is used for IO tasks, such as Direct Memory Access (DMA), device discovery, or error reporting. This sub-protocol is essential for every CXL device.
- **CXL.cache** enables a device, such as an accelerator, to cache data located in host memory. It uses cache-coherence mechanisms to ensure that locally cached copies remain consistent. This allows an accelerator to read and modify host memory while maintaining coherence with the CPU.
- **CXL.memory** enables processors and accelerators to access memory that is physically attached to a CXL device. This protocol is the main mechanism used for CXL-based memory expansion and allows an external memory device to be exposed as an additional memory region to the system.

CXL abilities. CXL provides several abilities that make it suitable for memory expansion. It offers additional memory that is independent from the CPU and GPU, adding to the total amount of memory available to the system. This memory can be accessed through DMA without requiring the programmer to manually split the data into chunks and copy them between CPU and GPU memory as in the copy-then-execute approach. This memory is also cheap compared to the GPU High Bandwidth Memory (HBM) as it is using affordable DDR memory.

2.4 CXL for GPU memory expansion

CXL-GPU overview.

CXL memory could be used to expand the GPU available memory pool. On the Figure 2.3 it is visible that the CXL acts as a unified memory between the CPU and the

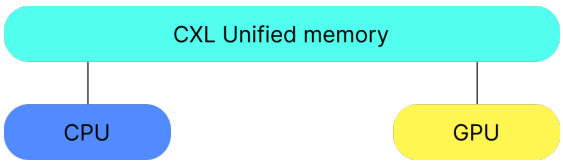


Figure 2.3: CXL memory as unified memory architecture

GPU. Moreover, GPU could access any data that is stored in the CXL memory, as CXL offers cache line access to any address.

Advantages of expanding GPU memory with CXL. The main advantage of CXL is that it can increase the amount of memory available to the GPU without requiring additional VRAM to be physically added to the GPU. CXL also supports cache line memory accesses, consequently making the CXL memory easier to use. Furthermore, CXL memory can potentially be shared between the CPU and GPU, allowing the same memory resource to be used by both processors. This makes CXL particularly interesting for applications where the required dataset is larger than the available GPU VRAM.

Consequential issues. Despite these advantages, CXL compared to the GPU's local VRAM has higher latency and lower bandwidth, which means that applications would experience a significant performance issues. Moreover, programmers may therefore still need to consider which data should remain in local VRAM and which data can be placed in CXL memory. This still leas to the search for performance optimizations and programming model simplifications.

2.5 Comparison

In the Table 2.1 it is clearly visible that CXL-integrated GPU does not have any extreme drawbacks. It is not as expensive as the NVLink-based solution, not as complex as copy-then-execute model and has better performance than UVM.

Table 2.1: Comparison of GPU interconnect / memory approaches

Approach	**NVLink**	**copy-then-execute**	**UVM**	**CXL-GPU**
Price	Extremely High	0 (software)	0 (software)	Low
Usage Simplicity	Simple	Complex	Simple	Medium
Total latency	Low	Medium	High	Low
Bandwidth	High	Medium	Low	Low
Memory type	HBM	HBM	DDR	DDR

3 Design

3.1 The Problem

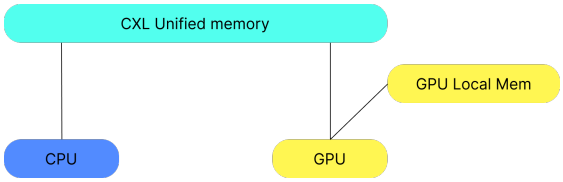


Figure 3.1: CXL-GPU

The problem with using CXL memory for a memory expansion for the GPUs, as discussed in the section 2.4, is not only the performance issue, but also the fact the programmer would need to decide where the data should be stored, in CXL or in the GPU local memory.

Problem statement: *Is it possible to leverage GPU memory transparently in order to reduce the impact of the CXL low bandwidth and high latency on the end system?*

3.2 The Proposal

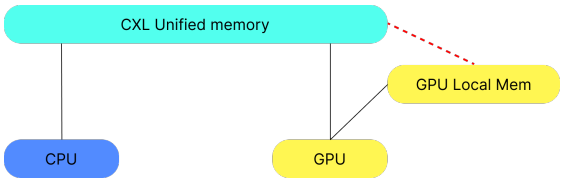


Figure 3.2: CXL and GPU memory connected

Overview. We propose a new hardware architecture in which GPU local memory is leveraged to *hide* CXL memory heterogeneous characteristics. The broader aim is to effectively flatten the two memory tiers (CXL and GPU local) through a new hardware abstraction.

Key idea. As CXL offers *cache-coherent* cache line accesses from the GPU, we extend the coherence mechanisms at the interface between the GPU and the CXL fabric to maintain coherence with the GPU local memory as well. In effect, our extensions keep the CXL unified memory and GPU local memory mutually exclusive, such that the GPU can retain a larger data set in its local memory and avoid CXL traffic, without changing access semantics from the CPU or GPU's perspective.

Benefits.

Transparent data management: If we could create such a system, then the first and foremost benefit would be that there would not be a need to move any data from CXL memory to GPU memory manually as it would be copied automatically. This is important, because now in order to use CXL memory with GPU effectively programmers need to copy data themselves. Moreover, at the current state of the system programmers need to decide, where to store the data, in the CXL memory or in GPU memory, but if we could implement the proposed idea, the data location would be selected automatically, which would also simplify the development of applications that use CXL memory.

Increased data locality: Furthermore, the total latency of the execution would be reduced, due to the fact that after the data was read by the GPU from the CXL memory, a copy of that data would be stored inside GPU local memory. The next loading of the data would be resolved at the GPU local memory, thus reducing single request and total latency, and also saving the bandwidth.

Correctness: Finally, as CXL memory is coherent with the CPU and the GPU out-of-the-box, there is no need to develop a completely new protocol that would ensure coherency between CPU and GPU sides.

3.3 Design goals

Our design goals are following:

- Generality
  - CXL memory is accessible through direct cache line access
- Efficiency
  - Low-latency reads of highly accessed data
  - Minimal bandwidth traffic to CXL
- Transparency

- No manual data copying required
- CXL latency should not cause GPU idling

3.4 Challenges

We have identified the following challenges:

- Integrating CXL client in the GPU side
- Design of a cache hierarchy that avoids necessary write-backs and improves cache-locality
- Ensuring that the new GPU cache hierarchy stays coherent and consistent with respect to the GPU and CPU programming models

3.4.1 CXL client integration

Integrating a coherence controller that is able to translate GPU requests into CXL requests and on the opposite accept CXL responses and restructure them in GPU memory responses could be a challenging task. To begin with, it would be hard to integrate a CXL client in the GPU memory, so the design shown at the figure 3.2. On the other hand, it would be better to integrate the client to the L2 GPU cache. (This way, we would also be able to change how the data request is handled in the cache and redirect it to the CXL memory or to the GPU memory based on the availability of memory in the local copy.) As this has already been achieved by the Victor Trost in his Bachelor's thesis, we will continue to expand his implementation.

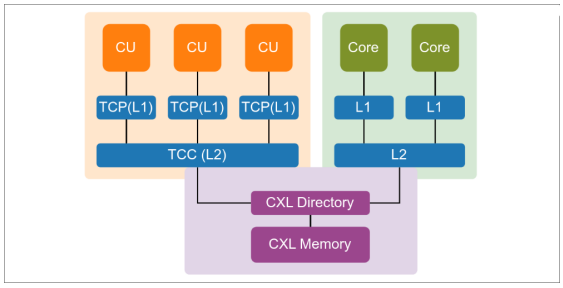


Figure 3.3: TCC by Victor Trost

In the figure 3.3 [19] a new module is introduced, Texture Cache per Channel (TCC). TCC is responsible for the translation of CXL and GPU cache coherence messages and it also serves as a cache between the GPU cores and the CXL memory.

3.4.2 Design of cache hierarchy

In the figure 3.4 the block called L3 is the level 3 cache, between TCC and GPU memory. This block is responsible for storing the data that was dropped from the TCC and send it further, once the L3 is full. L3 drops it's lines to the GPU memory, thus creating a hierarchy that was discussed in the proposition section.

3.4.3 Guaranties of coherency and consistency of CPU and GPU sides

As the L3 and GPU memory are only used for storing the data read from the CXL memory, and do not have any impact on the data that is being written by the GPU, the copy of the data in the CXL memory stays both consistent and coherent.

3.5 Architecture overview

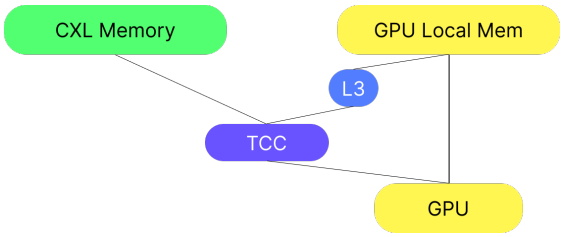


Figure 3.4: System overview

Figure 3.4 shows a high-level overview of the developed system: TCC is a bridge-cache, that requested by the GPU in the case of GPU core's cache miss. TCC is able to load data from the CXL memory, and then it stores it in it's own cache space. When the data is requested next time, TCC will not issue a request to the CXL memory, but return data that is stored locally. When TCC cache is full a victim will be elected on the Least Recently Used (LRU) principle and then it will be stored in the L3 cache.

L3 cache is a cache between the TCC and GPU memory. It is used as storage of cache lines that were sent from the TCC. When the storage of L3 gets full a victim will be selected on the LRU principle and it will be stored in the GPU memory.

As shown on the figure 3.2, L3 can access GPU memory in order to store data there, or to read the data from the memory.

3.6 System workflow

This section describes the workflow of the system. Our system should support two main operation: **Load** and **Store**

3.6.1 Store

When during an execution a GPU sends data to the TCC, TCC will attempt to store or update the data in the cache. In the case, when TCC cache is full, TCC will have to free space for the new data. This process is called eviction. [4] The evicted data will forwarded to the L3 cache, and the cache will store it, however this cache could also be full. This will cause another eviction, but this time data will be evicted to the GPU memory. by this principle, GPU memory and the L3 cache are going to be populated during runtime.

3.6.2 Load

If L3 receives a read request from the TCC, the first lookup place is the local cache space. If the data is found, then it will be sent back to the TCC and it will also be removed from the L3, because TCC will store this data now. If, on the other hand, data was not present, then L3 will issue memory request to the GPu memory. Memory is always slower then caches, so the L3 will have to wait for the reply. The reply will also be translated in the coherence message and returned to the TCC. L3 does not store any data, because TCC will store the data.

3.7 Summary

To sum up, with the system that we have described before, we will meet all the goals that were set in the beginning:

- Generality - CXL is accessible from the TCC, but it will only be accessed in case of TCC, L3 cache and GPU memory miss.
- Efficiency - CXL is accessed only once, after the first request, after that, the data will be stored in the TCC, L3 or GPU memory.
- Transparency - data-accesses are handled by the TCC and memory is loaded dynamically either from CXL or from local copy in GPU memory. Due to this user doesn't need to copy data manually and after first read latency equals GPU local memory latency.

3.8 Technical background

Building a real hardware module is beyond the scope of this thesis. However, how then would we test if the suggested system is working or showing results that are positive enough for the industry to consider implementing such optimization in the production systems? We will run the simulation of such system in the simulator called *gem5*. Gem5 is widely used in researches as it allows for simulating of custom systems, while maintaining realistic results. [18]

4 Implementation

4.1 Basic operations

As discussed in the design section there are two main operations that L3 has to support for successful operation of the system: Load and Store.

4.1.1 Store

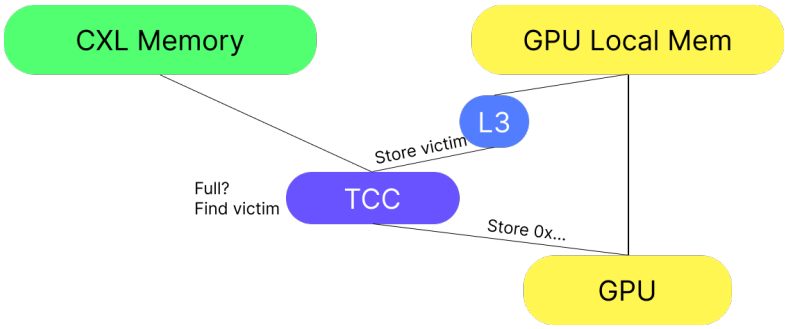


Figure 4.1: Storing in TCC

When L3 receives a store request it will check if the are any available space in the cache, in order to store the data that it just received. If it does have space, then the data will be stored and marked a most recently used. This is important for eviction process as the least recently used line will be removed. This way we ensure that the most relevant data stays as near to the GPU as possible. After the data is stored in the L3, an ACK will be sent back to the TCC. This process is illustrated on the Figure 4.1.

It is also possible that L3 cache is full. A victim will be selected and this victim will be sent to the GPU memory. Then, as there is a new free space, received data will be stored in that place and L3 will again send an ACK. This part process is illustrated on the Figure 4.2.

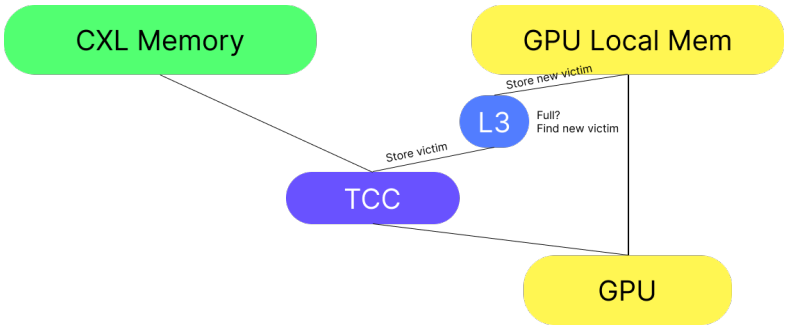


Figure 4.2: Evicting from L3 cache and storing in HBM

4.1.2 Load

When L3 receives a load request it's memory is checked to determine whether the requested memory line is stored locally. If this is true, then L3 cache would respond to the sender with the valid data. The data will be marked as invalid, so that the space could be reused later to store other data. This is done so, because TCC stores the most recently used data in order to ensure that GPU does not wait for extra cycles, while requesting the same data over and over again.

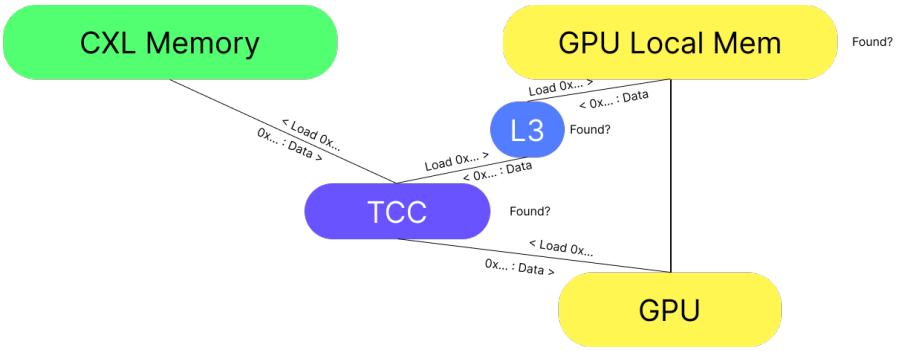


Figure 4.3: Full path

As L3 cache has a small storage, it is also possible that the data is not present in the cache and that means that the data has to be loaded from the GPU memory. The GPU memory would be populated through storing process described earlier. GPU memory is slower than caches and it takes some cycles to process the request, so the cache has to wait for the response from the memory. In order to replicate that in the gem5 simulator we have create a transient state. Entering such state indicates that the cache is currently waiting for the data. When the data arrives and the cache is in the

transient state, we can forward the data to the TCC. The data is not stored in the L3 cache, because it will be stored in the TCC as the most recently used, analogous to the previous paragraph. Lastly, if the data is not found even in the local memory, the request to the CXL memory will be issued. Such path is visible on the Figure 4.3

4.2 Asynchronous write-back

During the runtime of the program, due to the amount of the memory that has to be loaded in the GPU, the last layer of the described architecture, namely GPU Local memory, will eventually get filled with the evicted cache lines. At the same time, the memory that is fetched from the CXL memory and update by the GPU, could potentially be requested by the CPU. Hence, we need to ensure that the CPU will not read the outdated value.

This is achieved by the asynchronous write-back technique. When the cache line arrives at the TCC cache, it will store it an send an ack as soon as the data is processed, that, however does not mean that process of storing is completely finished. The rest of transactions that are described in the **Store** section will happen asynchronously, meaning that the GPU will not be waiting till all transactions have been completed.

4.3 Address translation

Both, GPU and CXL memory are assigned their unique memory spaces, that do not overlap. This is done so in order to ensure that the location of the memory line is deterministic across all available devices. That means that when the TCC receives a request, the TCC will also be able to determine, whether the memory is a locally created variable or it is a part of context that is stored in the CXL memory. On the other hand, this will be an issue, because during the store request processing the L3 will be storing the data from the CXL memory, and later evict it to the GPU memory, which, upon receiving a request, would attempt to store the data to the address from the wrong memory space. That is the reason, we have decided to introduce memory offsetting.

When a L3 is storing the data in the GPU memory, it will first change the first byte of the memory address to the predefined value. This way we ensure that the memory ranges of GPU memory and CXL memory do not overlap and the GPU is able to store the data. In the gem5 simulator we assign the specific range to the GPU memory, that starts with the byte of our selection. The Slicc, the language in which the coherence protocol is implemented in, unfortunately does not allow for the simple

address modifications, consequently, in order to achieve the required result, we need to modify the common library.

For this reason we have created a function *setMemoryOffset* that takes two arguments: the memory and the offset. The memory in gem5 is represented by the uint64_t, thus in order to set the first byte of the memory address we simply set it with the bit mask. We use this function in 3 places: when the read or write request is being send from the L3 to the GPU memory, and when the L3 is awaiting the data from the GPU Local memory in the transient state. The last case is different from the first two, because there instead of setting an offset we have to remove it.

4.4 Automatic data balancing

The core principle behind this design is that the system manages data balancing automatically, without requiring any manual data copying or moving from the programmer. The most relevant data is stored in the TCC, as this is where it can be accessed with the lowest latency. Data os lower relevance is moved to the L3 and to the GPU memory during the runtime. When the data from the L3 or GPU memory is requested by the TCC, it will be sent back into the TCC and removed from the L3, thereby restarting the balancing cycle. In the event that the requested data is present in neither the TCC not L3 or GPU memory, the TCC will fetch it from the CXL. The main benefit of this approach is that data relevance determines access latency, the more relevant a piece of data is, the fewer cycles are required to retrieve it. Consequently, the most important data is always the fastest to access.

4.5 Module integration

In order to integrate the L3 cache in out system, we modify the *GPU_VIPER_CXL_MESIF* file. In this file we have a TCC module created and connected to the other modules such as GPU and CXL memory. We add a new class to the configuration file, in which we instantiate all the parameters that are required by the Gem5 system. Furthermore, we add our own custom parameters, such as the offset to be able to change it from a single place in the system, a switch to be able to turn the L3 off. This will allow us to test the implementation with L3 or without, which will help us determine if there is an argument for an additional cache level between the TCC and GPU memory or against.

We do not create a separate class for the GPU memory buffer, that the cache will connect to. Instead of that, we use an already created module called HBMCtrl. We use exactly it, because this is the controller used in the system, to simulate the GPU memory too. This way we can ensure the correctness of the simulation results. The

HBMCtrl is an HBM controller that takes in the memory range as an argument and exposes the connection ports. This is useful, because in order for the address translation to work properly, we need the memory rage to begin with the offset byte that we would choose. Since, the HBM controller in this particular case would only represent the buffer that we have selected on the GPU memory, we can have it as a field in the L3 controller and not a separate module registered in the simulation system.

Due to the fact that the TCC is created as another module and the HBM is a submodule of L3, the connection of ports differs between the two cases. The TCC is a module that is registered in the simulation system and the connection will happen through the system connection ports. To make it possible we assign wirtual network numbers in the *slicc* implementation of the module, as well as the network type and the type of request that will be transmitted through that network. Since the network ports in the TCC slicc implementation have been assigned the same network number, the simulator would be able to transmit packets from one module to another without any manual intervention in the configuration part. The connection to the HBM memory, on the other hand has only defined the in and out ports, without networks ids. Moreover, since the created HBM controller is not registered in the system, we will have to connect it manually.

5 Evaluation

5.1 Research question

In the Chapter 3 we have stated that we want to create a system that reduces the total latency, by using the GPU local memory in order to reduce the impact of the latency and bandwidth problems of the end performance of the system. In the Chapter 4 chapter we have described the system that acts accordingly. In this chapter of the thesis we want to evaluate the results that we were able to achieve.

5.2 Evaluation Setup and Methodology

We use the version 25.0 of the gem5 simulator, which we run in the docker container provided by the ghcr.io. The configuration of the implementation, that we use, is in the apu_se.py. This is the configuration that was modified by the Victor Trost in his thesis [19]. We modify it further by adding the instance of the L3 cache, and we connect it's pins to the HBM controller. This controller represents a buffer in the GPU local memory, that would be reserved for our optimization purposes. We can use the system without explicitly allocating memory in the GPU local memory and just creating a separate module of the same memory type, due to the fact that this is a simulation and the parameters such as latency would be the same, thus this will have no impact on the test results.

In our system we use defaults mostly. Values that we have explicitly changed are:

- The amount of CPU cores is set to 16.
- The amount of GPU cores (CU) is set to 16 too.
- Total memory is set to 8 GiB.
- The GPU buffer is set to 2 GiB.

The rest such as latencies of different communication paths are test case specific and would be provided in each test case description.

5.3 Benchmark suite: Chai

As for the workloads that we will execute, we have selected Chai benchmarks [5]. These benchmarks are executing basic algorithms on the GPU. Chai benchmarks also provide support for multiple platforms, hence this benchmark suite is ready to be run in our simulation.

The main reason for implementing this system and conducting this research should be an increased performance, hence we will evaluate our system base on the time it takes to execute a workload. The gem5 simulator provides the stats.txt file after it has finished executing a workload. In that file there is the amount of tick that was simulated by the system, and this value is deterministic, meaning that even after running the workload multiple times on a different hardware would still yield the same result. We set this amount of ticks as a measure of the performance of the system and we will compare the amount of ticks it took our system to execute a benchmark to the amount of ticks of the system, without optimization of the CXL memory.

5.4 Performance comparisons

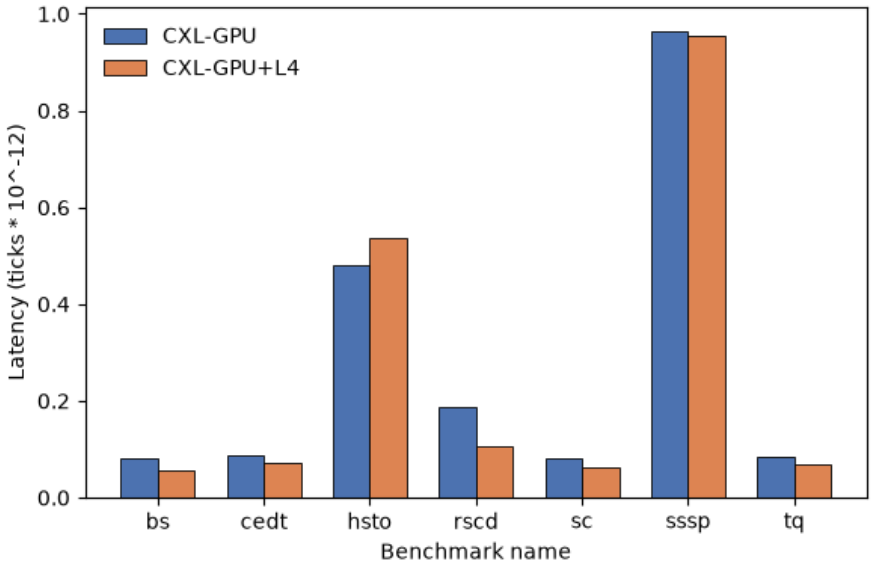


Figure 5.1: Chai benchmarks

In the Figure 5.1 we present the result of Chai benchmarks. The orange columns are representing the amount of ticks that the simulation has counted during the execution

in the system with the optimization. THe blue columns are, consequently, representing the system without. We can see that in the most of the scenarios the system that is caching the data in the GPU local memory has yielded a better performance, then the system that does not. This proves that caching data in the GPU memory will be beneficial for most of the workloads. That means that we have successfully designed and implemented the system that is capable of efficiently utilizing all the available resources in the system and that the CXL memory is a possible solution to the lack of GPU memory problem described in the introduction.

We can also see that the benchmark "hsto" was more efficient without the data being evicted to the GPU memory. This only means that the workload is designed in a way that is working against the caching abilities of the system. For example, a workload in which the CPU actively fetches the data from the CXL memory would show worse results, than the workload that does not. When a CPU fetches data from the CXL memory, it could happen that the data was overwritten and stored, for instance in the GPU local memory. In order to return a valid and not an outdated data, the CXL controller would send a request to the TCC cache, as it would recognize it as the owner of the data. Then the TCC would try to fetch the data from the L3, which would fetch it from the GPU memory. One such request has traveled across multiple objects effectively reducing the performance of the system.

5.5 Cached and uncached system performance

On the Figure 5.2 we are presenting the comparison of the total amounts of tick's the L3 cache enabled and disabled. We can clearly determine that the system with the L3 cache enabled is more performative than if that cache was disabled. Consequently, it would be beneficial for the system to be able to cache frequently accessed data in the dedicated fast cache space instead of always awaiting for the GPU memory latency, which is considerably higher than the one of the cache.

For most of the benchmarks removing cache did not have a huge impact. The most noticeable change was for the **rscd** benchmark. It's runtime was increased almost twice as much. This means that the rscd has a lower amount of the data that should be cache in order to increase the performance. The rest of the benchmarks relies on the L3 cache less. This is visible by the lower difference between cache-enabled and cache-disabled versions. This is due to the benchmarks not reading the same data multiple times. It fetches the data to the TCC from the CXL memory for it later to be evicted to the L3 and GPU memory, where it will stay. The performance increase of the cache-enabled version is due to the workload using the L3 cache in the beginning of the execution, when the cache is not yet full with the data. This, however, will not be true for most of

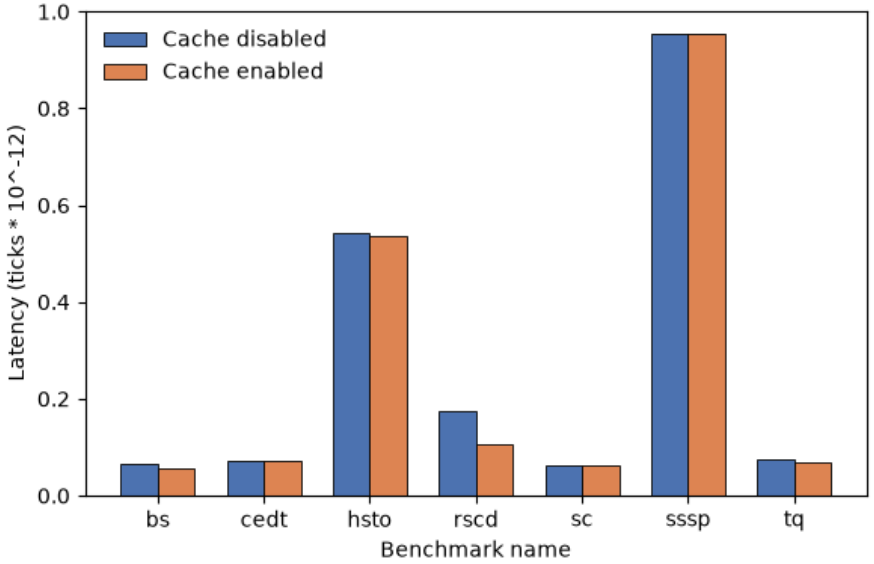


Figure 5.2: Cached and uncached results

the real world workloads, as these typically have a set of frequently accessed data that would stay in the TCC or the L3 due to the automatic data balancing described in the implementation section.

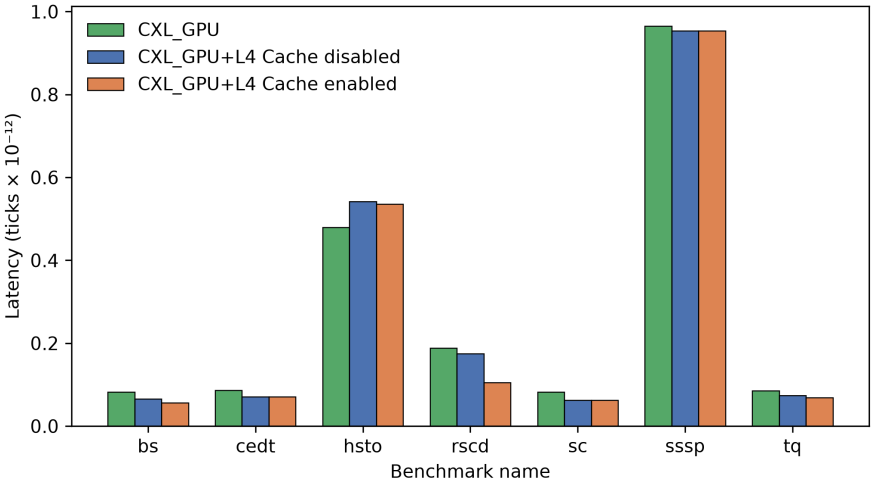


Figure 5.3: CXL-GPU, CXL-GPU+L4 Cache disabled, CXL-GPU+L4 Cache enabled

On the Figure 5.3 we also added the data from the version of the system that does

not offload data to the GPU memory. The comparison is important because now it is understandable that the system does not benefit from having more cache space, but rather from it's ability to utilize a part of the GPU memory to store the recently accessed data. On the chart we can see that the difference between the cache-enabled and cache-disabled versions are less noticeable as the difference in performance with plain CXL-GPU.

5.6 Baseline comparison

In this section we want to finish the evaluation with the comparison of three main systems:

- Baseline - the average CPU - GPU system that uses a dedicated model and copy-than-execute strategy.
- CXL-GPU - the system that is able to use CXL memory as coherent GPU memory expansion.
- CXL-GPU+L4 - the system that uses GPU memory to optimize the CXL memory.

In order to simulate the baseline system, we have taken the official unchanged gem5 simulator. All of the arguments such as the memory latency or the amount of the CPU or GPU kernels is set to match those in the CXL-GPU and CXL-GPU+L4.

5.6.1 Change of the benchmarking suite

As stated in the system requirements, we want a system that would let the programmer use the unified memory model. Both CXL-GPU and CXL-GPU+L4 correspond to this requirement, however since we want a correct comparison to the baseline system, this system has to use the dedicated model. Consequently, using the same implementation of the benchmarks would not yield correct results. Chai benchmarks conveniently provide us with the HIP-U-gem5 implementation, which translates to the: HIP framework, Unified memory model, suitable for gem5. In order to measure the baseline, we needed the version of the benchmarks that would be runnable in the dedicated memory model in the gem5. Gem5 only supports the ROCm stack, meaning we cannot use the CUDA framework.

The Chai benchmarks contain the CUDA-based version of benchmarks for the dedicated memory model. To be able to run the benchmarks in the simulator, we need to convert these benchmarks from using CUDA framework to the HIP. In order to achieve this we have used the HIPIFY tool. This tool is able to read the CUDA-based

code and output the HIP-based code, without changing the logic of the benchmark. It does is by finding and replacing the cuda-function with its analog in the HIP framework. After the hipification of all Chai benchmarks was finished, we have run into the same problem, that might have disallowed for the creation of the HIP-D-gem5 by the creators of Chai benchmarks.

The problem lies in the way the Chai benchmarks are designed. The work is distributed between the CPU and the GPU and both of them work in tandem to ensure the best possible performance. In order to ensure correct data processing without the race conditions, both devices rely on the atomic ints acting as flags to signal when the thread or a kernel may continue working on a task. This mechanism requires either a unified memory, which we do not have in the baseline model, or a mechanism to propagate an update to the rest of the system. For this purpose we would have used a function called atomicAdd_system(int* address, int value), which atomically adds an int value to the data stored by the given address. This function became available in the HIP stack from version 4.3.0, however the gem5 uses version 4.0.1. In order to be able to synchronize memory between the CPU and the GPU, during the GPU runtime, we would need to update the gem5 simulator to the next version.

Unfortunately, this task is not simple, since it would require reimplementation of parts of the gem5's core to suit the newer version of the ROCm stack. Since, this solution is beyond the scope of this thesis, we have decided to select a benchmark suite that does not require such operations.

5.6.2 Rodinia benchmarks

The rodinia benchmarks are another family of benchmarks that execute various algorithms on the GPU, such as breadth first search, gaussian algorithm and b+tree balancing. This benchmarks are also used in the scientific researches in order to measure the performance of different systems. Unlike the Chai benchmarks, the rodinia benchmarks are compatible with the ROCm of version 4.0.1, which makes that suitable to be run in our simulation. The rodinia benchmarks are also come with both versions for unified and for dedicated memory models, both of which are implemented using HIP frameworks.

5.6.3 Benchmark results

On the Figure 5.4 we are comparing three systems: CXL-GPU, CXL-GPU+L4 and the baseline system. Since we have already compared the CXL-GPU and CXL-GPU+L4 before the result that we are able to see on the figure is important as it is similar ot the one that we had before. The optimized CXL-GPU+L4 version is slightly faster than just

the CXL-GPU. Only the *lavaMD* benchmark shows a different result. This is still in order as we had faced the same result with the Chai benchmarks, where there was a similar case.

While comparing CXL-GPU+L4 with the baseline implementation, we can clearly see that in all benchmarks CXL-GPU+L4 has outperformed the baseline. This clearly indicates that the CXL-GPU has a real potential of being not only cheap, but also an effective memory expansion technology. Interestingly, the *lavaMD* benchmark was executed faster in the CXL-GPU+L4 than in the baseline, even though it was the only benchmark that had a decrease in performance compared to the CXL-GPU.

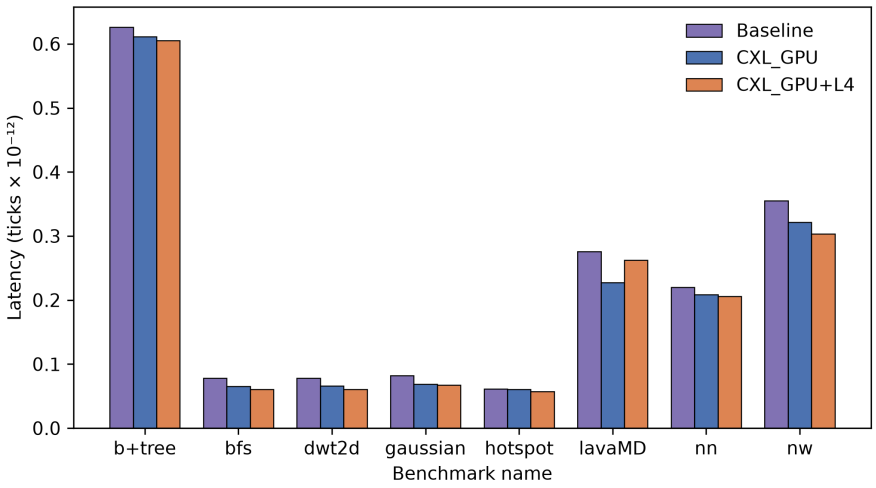


Figure 5.4: Systems comparison

6 Summary and conclusion

In the modern day world, developers and researchers often use the GPU in order to accelerate the workloads that the CPU is not optimized for. The critical point and the main bottleneck in the GPU for most of such workload has always been it's memory. Now, with the rise of Ai, we can clearly see the ongoing memory crisis, which makes us wonder: what if it was possible to expand the memory that is available for the GPU?

We propose using CXL memory as a memory expansion opportunity to effectively expand the available GPU memory capacity. Furthermore, we developed a system that optimizes the utilization of CXL memory by taking advantage of the GPU's local memory. To achieve this, we introduce a new level of cache that manages data movement between the GPU memory and CXL memory. This cache is designed to store data that is evicted from the CXL cache while also allowing for the accessed data to be stored in GPU local memory. Moreover, all of the new modules added to the system are ensured to keep cache coherency property, thus letting other devices of the system read correct and update data whenever it might be needed.

We have implemented the discussed system in the gem5 simulator that allowed us to simulate the hardware, while running the same software that could be run on the real hardware. This way we can ensure that our system will not be efficient only in the simulator and turn out to be a less efficient solution, when built and tested in the real world. We have used different benchmarks suites, designed independently from each other, and during the reviewing of results the system showed similar results for both of the benchmarks sets. Furthermore, the implementation of such benchmarks, was using the unified memory model, which is considered easier, when compared to the currently used dedicated memory model.

In conclusion, we were able to achieve a better performance than the plain CXL-integrated GPU, thus showing that the optimization of offloading data to the local memory would have a positive effect on the performance. Consequently, we have proofed that CXL memory is capable of providing a cheap memory expansion for the GPU memory, while also being a possible hardware-based optimization for the currently used copy-then-execute technology.

7 Related work

7.1 Disaggregated, Vendor-Generic APUs with CXL

In our research we have been relying on the work done in the research by Viktor Trost [19]. In his thesis, Viktor was able to integrate the CXL memory with both CPU and GPU. He has created bridges that were communicated with the CXL memory from each side. The relevant bridge for our research is the TCC, or L2 cache that is sable to resolve CXL memory requests. Viktor did mention the possibility of usage of the GPU local memory in order to space the bandwidth and decrease the average latency of the request in the Future work section.

8 Future work

8.1 Realistic workloads

In the evaluation part, in order to make statements on the models performance we have relied on the benchmarks that are used for testing such systems, however, as we have seen the performance varies from one benchmark to another. This is happened to the different approaches that are used in the benchmarks, which would also mean that the real workloads, could use the approaches that were not considered by this benchmarks. Moreover, these test usually use the data sets that are not comparable to the real ones. For this reason it would be important to test the suggested system against real world workloads to determine, whether it would be truly beneficial and not solely in the simulator.

8.2 Newer versions of ROCm

As mentioned in the evaluation part, the gem5 simulator uses an outdated version (4.0.1) of ROCm hipcc. As the consequence of that, we were not able to run benchmarks that relied on the runtime updates from the GPU to the memory that is read by the CPU. If the backend of the simulator was updated to support the latest version of the ROCm stack, we would be able to run far more complicated benchmarks, hence the system would be tested with more possible use-cases for vendors to decide whether to employ it or not.

Abbreviations

CXL Compute Express Link

TCC Texture Cache per Channel

LRU Least Recently Used

CPU Central Processing Unit

GPU Graphical Processing Unit

UVM Unified Virtual Memory

PCIe Peripheral Component Interconnect Express

DMA Direct Memory Access

HBM High Bandwidth Memory

GPGPU General-purpose computing on GPUs

List of Figures

2.2 Copy-the-execute example 7
2.3 CXL memory as unified memory architecture 9

3.1 CXL-GPU 10
3.2 CXL and GPU memory connected 10
3.3 TCC by Victor Trost 12
3.4 System overview 13

4.1 Storing in TCC 16
4.2 Evicting from L3 cache and storing in HBM 17
4.3 Full path 17

5.1 Chai benchmarks 22
5.2 Cached and uncached results 24
5.3 CXL-GPU, CXL-GPU+L4 Cache disabled, CXL-GPU+L4 Cache enabled 24
5.4 Systems comparison 27

List of Tables

2.1 Comparison of GPU interconnect / memory approaches 9

Bibliography

[1] *About CXL*. URL: https://computeexpresslink.org/about-cxl/.

[2] *Accelerate Compute-Intensive Workloads with NVIDIA-Certified Systems*. URL: https://www.nvidia.com/content/dam/en-zz/Solutions/gtcf20/certified-systems/nvidia-certified-systems-white-paper.pdf.

[3] M. Anderson. *Understanding GPU-Powered AI Workloads*. 2025. URL: https://medium.com/@mike.anderson007/understanding-gpu-powered-ai-workloads-8a74f99d1ad8.

[4] *Cache Eviction*. 2026. URL: https://www.geeksforgeeks.org/system-design/cache-eviction-policies-system-design/.

[5] *Chai benchmarks source code*. URL: https://github.com/chai-benchmarks/chai.

[6] *CRIUgpu: Transparent Checkpointing of GPU-Accelerated Workloads*. 2025. URL: https://arxiv.org/pdf/2502.16631v1.

[7] *DeepSeek V3 Overview*. URL: https://deepwiki.com/deepseek-ai/DeepSeek-V3.

[8] *Embarrasingly parallel*. URL: https://en.wikipedia.org/wiki/Embarrassingly_parallel.

[9] *GPT-3 Overview*. 2022. URL: https://deepwiki.com/openai/gpt-3.

[10] *GPT-2: 1.5B release*. 2019. URL: https://openai.com/index/gpt-2-1-5b-release/.

[11] M. Harris. *Unified Memory for CUDA Beginners*. 2017. URL: https://developer.nvidia.com/blog/unified-memory-cuda-beginners/.

[12] Mitrasish. *What Is Shared GPU Memory? Dedicated vs Shared*. 2026. URL: https://www.spheron.network/blog/dedicated-vs-shared-gpu-memory/.

[13] S. Mukherjee. *Understanding Parallel Computing: GPUs vs CPUs Explained*. 2024. URL: https://www.digitalocean.com/community/tutorials/parallel-computing-gpu-vs-cpu-with-cuda.

[14] *NVIDIA CUDA*. URL: https://docs.nvidia.com/cuda/doc/index.html.

[15] *NVIDIA NVLink*. URL: https://www.nvidia.com/en-us/data-center/nvlink/.

[16] *NVIDIA-Accelerated Data Science*. URL: https://www.nvidia.com/en-us/deep-learning-ai/solutions/data-science/.

[17] *Simulation and modeling*. URL: https://www.nvidia.com/en-us/high-performance-computing/simulation-and-modeling/.

[18] *The gem5 Simulator*. 2020. URL: https://arxiv.org/pdf/2007.03152.

[19] V. Trost. *Disaggregated, VEndor-Generic APUs with CXL*. 2025. URL: https://github.com/TUM-DSE/research-work-archive/blob/main/archive/2025/winter/docs/bsc_trost.pdf.

[20] *Unified Virtual Memory*. URL: https://docs.nvidia.com/ai-enterprise/release-8/latest/infra-software/vgpu/features/uvm.html.